The Gazette

VOL. 1, NO. 008

2026-03-30

FEATURES

Ninety Days of Commits

Jake Yesbeck

It has been ninety days since I started the Year of Commits initiative. The criteria behind this challenge was to make a commit to a public Github repository every day. Technically, a commit like the following satisfies that criteria:

However, if I allowed this commit to be my sole contribution for a day, I would be cheating no one except myself. In reality, I have been averaging 3 to 4 decent size commits a day. Spending around an hour or two total per night. Github is an awesome piece of software that tells a visual tale of when I am most productive:

Over this quarter of a year, I have accrued some interesting learnings about developing open source software.

1. Writing software every day just feels good.

Not missing a single day of writing software has made a significant improvement in my confidence and attitude. In the same way exercising daily makes your body feel more sturdy, developing software daily keeps my mind focused and clear. Like warming up a car before a drive, writing software every day makes it very easy for me to jump into a new project or build a new feature. I think most of my fellow software artisans have been in this situation: You go on vacation or maybe just a very relaxing weekend, then the next day at work or when you open that side project again, you end up spending a lot of time getting familiar with the software. Even if you just wrote it a few days ago, the break has made some tiny details less sharp. Writing software every single day helps eliminate this.

2. Immature software is expected

While writing software, there are a total of 0 people that look at what they just wrote and say: “This software is already bad and I have only just written it”. However, this statement can be more true than we would like. The potential in a new project is always the best part, right? Developers around the world usually jump at a change to do green field development. After all, with no legacy software around we can make what ever our heart desires. It can be clean, DRY, clever software with all the best design patterns. But, at least in my experience, all software starts off painfully immature. This is just fine. The shape and structure of good software grows over time. This is particularly true for my Year of Commits projects. Since I must write software every single day, I revisit my software extremely frequently. This re-visitation greatly increases the iterations my software goes through and I end up with much more polished work.

3. Software development is not just making it work

If you take away business requirements, deadlines, conversion rates, etc., you are left with something pure: software development. Year of Commits has enabled me to experience developing software at a slower pace without these extra degrees of “purpose”. Because of this, I have learned the difference between software that just works and carefully architect-ed software. I am not boasting that the software I write is now amazingly designed and executed, I have just noticed that when I invest more time, it makes a difference. We, as Software Aristans need time to stop and smell the ones and zeros. Given a decent amount of time and attention, all software can mature to something really extraordinary.

I am optimistic about completing my goal of a Year of Commits. With 3/4 the distance ahead of me, let’s hope I can keep that graph solid green.

A Simple Laravel Hello World

Scott Johnson (fuzzyblog)

As of late I've been working in Laravel and figuring out how Laravel differs from Rails. And while I have a bigger blog post in the works, I thought there was learning value in a simple Hello World implemented in Laravel. Hello World is a simple program that just puts Hello World on the screen.

Here's how I thought a Laravel version of this should work:

Be a whole Laravel application

Be a single controller

Be a single route

Define the text "Hello World" as a variable, $text, in the controller

Display that on the screen via an HTML template

While this doesn't illustrated models and database connectivity, this actually illustrates how to use a surprising amount of an overall MVC framework.

Step 1: Creating an Application

The first step is to use composer to generate the overall application with:

composer create-project laravel/laravel hello_world

The next step is to change into the directory with:

cd hello_world

and then open the code base with:

mate .

or if you use Sublime:

subl .

Step 2: Generating a Controller

Back at your command line, you want to generate a controller with:

php artisan make:controller HelloController

This will create a file named HelloController in app/http/Controllers:

ls -l app/Http/Controllers total 16 -rw-r–r– 1 sjohnson staff 361 May 18 11:37 Controller.php -rw-r–r– 1 sjohnson staff 122 May 27 09:12 HelloController.php

Inside the controller you want to add the following code (some of which will already be there from the generator):

public function index () { return view ('hello_index'); }

inside the "class HelloController extends Controller" block of code (it goes inside the {} braces).

Step 3: Adding a Route

Moving back to the code base, you want to edit the file routes/web.php and add this line:

Route :: get ('/hello' , 'App\Http\Controllers\HelloController@index');

This adds a single route, /hello, which is then handled by the Hello controller and the hello_index action.

You can test that this route is available by running:

php artisan route:list

which should show you something like this:

+——+———-+———-+——+————————————————+———+ | Domain | Method | URI | Name | Action | Middleware | +——+———-+———-+——+————————————————+———+ | | GET|HEAD | / | | Closure | web | | | GET|HEAD | api/user | | Closure | api | | | | | | auth:api | | | GET|HEAD | hello | | App\Http\Controllers\HelloController@index | web | +——+———-+———-+——+————————————————+———+

Note : It is critical that you use the fully name spaced path to the controller (i.e. starting with App). According to what I've read, you can use a use declaration at the top of the routes file to import controllers so you don't have to have the name spacing everywhere but I was unable to ever make that work.

Step 4: Adding a Variable to the Controller

Inside the public function index method of the Hello controller, you want to add / change the following code:

This defines one variable, $text, which should then be available in the view. The way that it becomes available to the view is by passing a hash into the return statement which has a key of 'text' which is mapped to the variable $text.

Step 5: Creating a View and Displaying a Variable

Views, in Laravel, live in the directory resources/views so you can create one with the command:

touch resources/views/hello_index.blade.php

And you then want to edit that file and add this code:

Step 6: Previewing the Results

The key command here is:

php artisan serve

which starts the application and displays the result on:

Teaching the Old Dog a New Trick

Dainius Jocas · Vinted Engineering

TL: DR : When required data can be encoded with match-features, Vespa can apply a new optimisation, which can be a lifesaver when data is frequently redistributed.

Use-case

When ranking, Vespa requires additional information that cannot be easily stored within the documents being ranked (e.g. the statistical cross features between the user and the document such as a counter of how many interactions the user has made with the documents of this category can't be stored within an item itself). Then, you need to pass them as parameters via ranking features . An important question is: where do you store and fetch that information from?

When Redis became a bottleneck for this task, we decided to try Vespa itself. Why?

The data format is already suitable because it is going to be passed into the ranking profile.

It becomes possible to eliminate some network round-trips.

Vespa is scalable to any dataset size and can store data both in memory and on disk.

Vespa allows for fetching multiple filtered documents.

Vespa allows for collocating calculations with the data.

And of course, such tasks require single-digit millisecond latency.

Initially, the use e of Vespa for use cases worked well, and looked like a great success. However, the proverbial honeymoon ended when the number of schemas (50+) and the update rate (500M+ per hour) skyrocketed. We noticed that sometimes tail (p99+) latencies bumped to 100ms+, seemingly out of nowhere, but the bump sometimes correlated with the high feeding bursts (meaning right after the feeding burst). Such high latencies are unacceptable when the latency budget is 50 ms.

We noticed that the spike in tail latencies always happened after a burst of feeding requests. E.g. features that are recalculated hourly/daily for all Vinted users (i.e. 100+ million records) create such bursts. When latencies started spiking more and more frequently, it was a signal to have a closer look to see what the cause was.

The diagram above shows that a spike in p99 latency occurred after a feeding burst.

After inspecting the logs during such a latency spike, there were multiple records such as the following:

Docsum fetch failed for 36 hits (64 ok hits), no retry

The log says that some document summaries have failed.

After a guru meditation session at the Vilnius office sauna (which is intended for such type of work), we concluded that the data is moving around the cluster and it causes problems for document summary fetching.

This theory was quickly confirmed by checking the dashboard on data redistribution.

With this evidence, the problem to solve was clear but first, we need to familiarise ourselves with how Vespa executes queries.

This is the typical query execution flow:

The diagram above shows that a request first comes to the Vespa container node. Then, it is scattered to all content nodes (typically over the network) of an available content group. Responses with local Top-K hits are then gathered in the container node. The Top-K global matching documents a .fill() request is once again sent to the relevant content nodes to fetch the document summary (i.e. document data or calculated values).

When a data redistribution is ongoing during the query handling, it might happen that between the query execution and .fill() (that typically takes a couple of milliseconds), the documents is moved from one content node to another (or the content node is down, or some other unexpected situation that happens in distributed systems). To handle such a situation, Vespa queries all known content nodes for the summary data, potentially doing multiple retries .

Typically, summary fetching takes 1 ms, but we've seen summary fetching taking 100 ms.

A small nuance to note about the query execution flow is that, with the first response from content nodes, the match-features can be returned.

Match features are rank features, added to each hit into the matchfeatures field. The feature was added to Vespa in 2021 . The values can be either floating point numbers or tensors (but not strings, booleans, etc.). Typically, they are useful to record the feature values used in scoring for further ranking optimisation.

A clever trick to encode non-numeric data, e.g. a string label, is to convert it into a mapped tensor . If you squint a little, the mapped tensor looks like a regular JSON object.

By knowing the problem and being familiar with matchfeatures, we can draft a workaround for summary fetching.

Luckily, we've already thought about such an optimisation ! What if everything we needed could be fetched with the select match-features from ... ? Summary fetching would then not be required, and .fill() could be eliminated .

The enthusiasm led to a quick proof of concept; however, the benchmarks surprisingly showed no improvement at all. This led to an inspection of the query trace , in which

Validates Type 2.0

Jake Yesbeck

One of the first projects that I worked on during this Year of Commits was validates_type . In case that name is too obscure, validates_type is a gem for validating that a specific value is exactly the type it is expected to be. The validates_type library is compatible to ActiveModel style validations.

Originally, the validates_type gem was very strict with what it validated against. It started with basic included Ruby types like Integer , Float , and String . This was a fine first step but proved too restrictive for basically any use case.

In a recent update, the validates_type gem has been extended to validate against any defined type in a system. With this update, the uses of this gem greatly increased. They increased so much that it is possible more than 2 people will use it, and that would be pretty awesome.

The validates_type gem can be useful for a system that cares about exact types at save time. For instance, a model that has an incorrect database column type (because of legacy or other reasons) can still control validation over its data just the same.

If a “junior developer who is now somehow the CTO” created the original system. And if that developer did something like create an is_published column on the authors table set as a varchar , validates_type makes that less of an issue.

class Author > author = Author . new # => # > author . is_published = ‘foo’ # => “foo” > author . save! # => ActiveRecord:: RecordInvalid: Validation failed: is_published is expected to be a Boolean and is not.

This way, there is not a bunch of random data in the authors table because of a bad decision made a while ago. More examples on how this gem can be used are found at the prequel to this post and the project’s README .

The 2.0 update brought flexibility to validates_type . This enables greater control over serialized attributes on ActiveRecord objects.

In a system with an intermediary class inserted between the column assignment and the column serialization, this extension can show its true value.

In an application that deals with Books , each book must store information regarding how it was published:

class Book > book = Book . new > book . publishing_information = { foo: :bar } > book . save! # => UPDATE “books” SET “publishing_information” = $1, “updated_at” = $2 WHERE “books”.“id” = $3 [[“publishing_information”, “{}”], [“updated_at”, “2015-11-30 05:04:09.480707”], [“id”, 3]]

If, like the above example, the PublishingInformation class is forgiving, this will silently fail. The book.publishing_information will not be set to the correct value.

On the other hand, if the PublishingInformation does fail, it might not do so in an obvious way.

However, with validates_type , this issue becomes explicit.

Adding the type validation to the book model:

class Book > b = Book . new > b . publishing_information = { a: :b } > b . save! # => ActiveRecord:: RecordInvalid: Validation failed: publishing_information is expected to be a PublishingInformation and is not.

Now, a readable error is produced and nothing is left to the imagination.

A Reasonable Start

Was the used example extremely specific? Yes. But, I would wager that at least one application out there has had a problem similar to this and validates_type can ensure that it never happens again. Hopefully, this example and the problem it did end up solving with at least act as inspiration for this library’s other uses out in the wild.

I have found a use for this type of validation and hope others can as well. If a use case arises that this gem does not support but could easily, I invite everyone to contribute to it .

Haskell Researchers Announce Discovery of Industry Programmer Who Gives a Shit

Steve Yegge

The worldwide Haskell community met up over beers today to celebrate their unprecedented discovery of an industry programmer who gives a shit about Haskell.

On Wednesday, researchers issued a press release revealing that 27-year-old Seth Briars of North Carolina, a Java programmer at Blackwater accounting firm Ross and Fordham, actually gives a shit about Haskell.

“Mr. Briars has followed every single one of our press releases for years,” the press release stated. “Probably even this one.”

Haskell researcher Dutch Van Der Linde explained how they had stumbled on the theoretical possibility of Briars and his persistent interest in Haskell. “We knew that there are precisely 38 people who give a shit about Haskell,” said Van Der Linde, “because every Haskell-related reddit post gets exactly 38 upvotes. It’s a pure, deterministic function of no arguments – that is, the result is independent of what we actually announce. But there are only 37 of us on our mailing list, so we figured there was a lurker somewhere.”

“That, or it was an off-by-1 error not detectable by our type system,” Van Der Linde added. “But we don’t, uh, like to dwell on, I mean with good unit testing practices we can, um... sorry, I need to get some water.”

As Van Der Linde stumbled off in a coughing fit, his fellow researcher Bonnie MacFarlane outlined their basic dilemma: “Finding a person who gives a shit about Haskell is an inherently NP-complete computer science problem. It’s similar in scope and complexity to the problem of trying to find a tenured academic who didn’t have the bulk of his or her work done by uncredited graduate students. So even though we suspected Briars existed, we needed a strategy to smoke him out.”

She explained the trap they set for Briars: “We crafted a fake satirical post lampooning Haskell as an unusable, overly complex turd – a writing task that was emotionally difficult but conceptually trivial. Then we laced the post with deeper social subtext decrying the endemic superficiality and laziness of global industry programming culture, to make ourselves feel better. Finally, each of us upvoted the post, which was unexpectedly contentious because nobody could agree on what the reddit voting arrows actually mean.”

“And then we waited to see who, if anyone, would give a shit,” she said.

MacFarlane concluded, “Our elegant approach didn’t work, so we hired a Perl hacker to go dig up the personal details on all 38 accounts that had ever upvoted a Haskell post, and the only one we didn’t know was Seth Briars. So we reached out to him, and thankfully so far he hasn’t threatened to sue us.”

Briars says he is pleased to have been recognized for his apparently unique shit-giving about Haskell. “I’ve been giving a shit about Haskell for a long as I can remember. I follow all their announcements and developments closely, just in case I ever get the urge to use the language for something someday.”

“It’s a beautiful, elegant language,” Briars observed as he busied himself cleaning a fingernail. “You’d be hard-pressed to find a more expressive and composable core. And they’ve made astounding advances over the years in performance, interoperability, extensibility, tooling and documentation.”

“I’m kind of surprised I’m the only person on earth who gives a shit about it,” Briars continued. “I’d have thought there would be more people following the press releases closely and then not using Haskell. But they all just skip the press releases and go straight to the not using it part.”

“People see words like monads and category theory ,” Briars continued, swatting invisible flies around his head for emphasis, “and their Giving a Shit gene shuts down faster than a teabagger with a grade-school arithmetic book. I’m really disappointed that more programmers don’t get actively involved in reading endless threads about how to subvert Haskell’s type system to accomplish basic shit you can do in other languages. But I guess that’s the lazy, ignorant, careless world we live in: the so-called ‘real’ world.”

Haskell researcher Javier Escuella remains hopeful that one day they may be able to double or even triple the number of industry programmers who give a shit about Haskell. “I believe the root cause of the popularity problem is Haskell’s lack of reasonable support for mutually recursive generic container types. If we can create a monadic composition-functor wrapper that is perceived as sufficiently sexy by hardened industry veterans, then I think we will see an uptick in giving a shit, possibly as much as a full extra person.”

Haskell aficionado Harold MacDougal is not quite as sanguine as his colleague Escuella. “I doubt Haskell will ever be appreciated by the uneducated natives of this industry. As exciting as it is, the discovery of Briars should be considered an anomaly, and not as a sign that more people will ever give a shit. Programmers only seem to pay attention to things when there is humor involved.”

“We do have an experimental humor monad,” added MacDougal. “But it doesn’t seem to be getting much adoption. Haskell fans just don’t see the need for it.”

MORE NEWS

Previous article: Perl Community Debating Adding Monads
The Perl lists are brimming with discussions about the value of adding monads to Perl. “We don’t really know what they do, but it doesn’t make sense _not_ to have something in Perl,” said Perl hacker Landon Ricketts. Read more

The AI boom has plunged a small Pennsylvania town into chaos

Rebecca Egan McCarthy · Grist

"I don't like to see anyone upset," said Nick Farris of Provident Real Estate Advisors. He was sitting in the front of a crowd of roughly 150 inside Valley View High School's auditorium in Archbald, a town of about 7,500, huddled between two mountain ranges in Pennsylvania's Lackawanna Valley. Farris was there to represent the developer for Project Scott, one of many data center campuses coming to town. "However," he said. "I think that this is the best data center site in this area of the country, by far." The audience had been fairly quiet, bundled in thick coats against the late January cold. But as Farris spoke about data centers as a boon for communities, they began to laugh, drawing a rebuke from town officials.

"What about the children?" someone shouted from the crowd. The children were watching from the walls; long banners of Valley View Performing Arts students hanging around the auditorium like championship pennants. Project Scott and four other data facilities will sit just a few thousand feet from the middle and high schools.

"Isn't there a missile plant next door?" Farris said, getting aggravated. He was referring to Lockheed Martin's 350,000-square-foot Missiles and Fire Control facility directly next to the high school, parts of which are highly contaminated .

"That sucks too!" another attendee yelled back. This was nothing to worry about, Farris tried to convince the audience. This would bring in tax revenue, he said. It was just an office park, albeit one with roughly 450 diesel backup generators.

"It's going to be away from everyone," Farris kept repeating, to rising jeers. He was wearing a knit turtleneck with a large American flag emblazoned across the chest. "It's not going to bother anyone."

The specifications say something different: Five developers are planning to build six data center campuses in Archbald, which will cover a full 14 percent of the town, evict a trailer park, and border many residential properties. One campus alone, as The Scranton Times-Tribune reporter Frank Lefneskey pointed out, is expected to use more power than the region's largest power plant is able to produce.

Pennsylvania has become an epicenter of the data center boom in the United States, with over 50 campuses in development. Eleven of them are slated for Lackawanna County alone. Archbald, with six campuses composed of 51 massive buildings, has the most of any municipality in Pennsylvania.

Despite the public outcry, it has been surprisingly difficult to learn what Archbald's elected officials think of the massive industry moving in. None of the town's seven council members responded to my emails, so I stopped by the borough administration building in person a few weeks before Christmas, and was told to wait in the lobby while they held an informal closed meeting in council chambers. When the door opened, it was clear that anyone who actually had the power to make or break the data center plans had quietly filed out the back, leaving me with Archbald's unelected borough manager, Dan Markey, who essentially runs the town, but cannot vote for or against development.

What did he think about artificial general intelligence, or AGI — the idea that eventually these efforts will produce something like a "computer god" capable of solving climate change, ending hunger, revealing the full breadth of science, and performing any number of other miracles? "I believe in one God, and it's not a computer," Markey told me evenly. By now, hundreds of towns across the country have been caught up in the rush to build large language models that can parse unfathomable amounts of data to write copy, answer any query, develop new vaccines, and likely render a large number of jobs obsolete. That rush accelerated into an all-out arms race over the past year. What it means in practice is an enormous amount of data centers — enough to approximate a minor deity and enough, as OpenAI cofounder and former chief scientist Ilya Sutskever once speculated, to "cover the Earth." Markey told me he had ChatGPT on his phone, but didn't use it much.

"I don't think anyone in their right mind wants to see the world covered in data centers," Markey said. "[But], according to Pennsylvania law, we have to have a zone for everything. An adult bookstore, a strip club, a concrete and asphalt plant — anything that wants to come here. We have to have a zone for it. If it's not zoned, it's allowed to go anywhere."

In most states, towns have the right to exclude businesses that they find disagreeable — a wealthy suburb, for example, would likely reject a landfill or gas plant — but in Pennsylvania, towns must allocate some patch of land to these "undesirable industries." Some municipalities deal with this by forming what are called zoning collaboratives, which allow them to plan as a region for pollution. One town might get data centers, another gas plants, another a landfill. Markey approached the nearby boroughs of Blakely and Dickson City to discuss the possibility, but the data center development rush has outpaced him.

Whatever's attracting data centers to the area, it's forced Markey to answer for a rush of development, unprecedented since the town was settled in the 1840s in service of an industry that will bring vanishingly few jobs. Residents kept bringing up the threat of collapse — the network of empty mine shafts running underground, the town beneath the town, the structural instability that would accompany these massive buildings. The data centers would drive bears into town, they said, rattlesnakes into yards, and without trees to stabilize them, the mountains themselves would begin to crumble, sending landslides into the valley.

"I just try to listen, and I try to separate the valid concerns from the things that just sound like the sky is falling," said Markey. "I was approached at a gas station a couple months ago and was told that I was going to kill everyone who lived

When Jupyter Notebooks Won't Import Libraries

Scott Johnson (fuzzyblog)

I don't claim to be an expert when it comes to Python. At best I'm an apprentice striving to be a journey man. One of the interesting tools in the Python / Data Science ecosystem is the Jupyter Notebook which gives a cell based representation of code, visualizations, documentation and execution flow and allows you to package things up for distribution i.e. hand your work, in a complete fashion, from say Data Scientist 1 ("Rebekah") to Data Scientist 2 ("Dawn"). This is a laudable goal and one that I theoretically agree with.

Note : Jupyter Notebooks do NOT include data so that's still external to the notebook, something that can easily bite you (as it is currently biting me).

I know a lot of pythonistas simply install libraries to their local machine and just have a collection of tools that the throw at problems. This, however, is a terrible practice due to code deprecation, version conflicts, etc. I say this with authority because I've been through this in the Ruby world before we all regularly started using Ruby virtual environments / Ruby version managers like RBENV / RVM which manage dependencies on a per project basis. Knowing this, my first practice with Python, is to always create a virtual environment, generally using Virtual ENV (VENV).

So, when I set out today to use a Jupyter Notebook, my first approach was to make this work with a Jupyter Notebook. And, alas, I haven't been so lucky as to make this work cleanly with a Python virtual environment like VENV. Here are some of the things I tried:

Installing jupyter into the virtual env and running from bin/ jupyter notebook

Changing the Kernel

If you're curious about how to use Python Virtual Environments, I wrote a solid tutorial back in September that I used to get a full installation of Tensor Flow up and running. I've referred back to this over and over, each time I needed a Python Virtual Environment, so I know it works.

I'm sure there is a way to mess about with virtual environments and Jupyter Notebooks to make them work but, honestly, I'm skeptical on notebooks and how they obfuscate code and data together anyway so I thought "How do I just make this a Python script". I took this approach because I was absolutely certain that I could make a virtual environment work with just Python. And, thanks to my pairing partner, Grant, there is, indeed, a way.

Making a Jupyter Notebook into a Python Script with a Virtual Env

Follow the instructions here to setup VENV for a new project in a new directory.

Use File menu, Export as Python to write a single Python script representing the notebook.

Create a requirements.txt file as per the instructions here .

Go through the Python code that was exported and convert the import / from instructions to the right PyPy package index name entry in the requirements.txt file. Be aware that there isn't a straight correspondence between the import statements and the package names. For example you import from "metal" in the notebook I'm porting but the package name is actually "snorkel-metal" and you import from sklearn but the package name is actually scikit-learn. Python's there's only one way to do this mantra, in the area of package management, is just plain crap. Sigh.

Run pip3 install -r requirements.txt

Run your python script and then adjust the requirements.txt file accordingly. You will almost certainly need to change some things but, by and large, I'm finding that this process does work.

Ask a Climate Therapist: How can I balance my travel itch with guilt about emissions?

Leslie Davenport · Grist

Dear Leslie,

Almost all of the money I save goes toward travel — meeting friends abroad, visiting relatives, or checking national and international parks off my bucket list. I try to be as sustainable in my personal life as possible, but I love traveling, despite knowing that travel is where some of our greatest carbon emissions come from. Every time I fly now, I feel this guilt and fear, and it's sometimes hard to balance it with my excitement. How would you recommend proceeding when some of the decisions we make in our personal lives are at odds with our beliefs and goals?

— Wondering Wanderer

Dear Wondering Wanderer,

The tension you're describing — the pull between loving this world and worrying about harming it — is a sign of a very intact compassion compass. Sometimes, that compassionate awareness stretches us beyond what's comfortable or easy to resolve.

Don't dismiss your discomfort, because it's inviting you to stay awake to your impacts on the planet — but don't let it shut down your capacity for joy. If guilt grows so heavy that it diminishes your ability to connect and feel alive, it's lost any usefulness it once offered. The work now is to metabolize your guilt and fear into something that guides you rather than pulls you apart.

Ask a Climate Therapist tackles your questions about how to navigate the emotional side of climate change, with leading climate-aware therapist Leslie Davenport. Have a question? Ask it here !

You might ask yourself this question: Given what I know, what kind of traveler do I want to be?

That opens up more nuanced choices than simply "Go or don't go?" Maybe you'll travel less often but stay longer, explore more locally using lower-carbon methods of transport, or prioritize trips that deepen relationships.

You might also focus on destinations where you can make a positive impact as a tourist. In countries like Costa Rica, Rwanda, Tanzania, and Bhutan, responsible tourism helps keep conservation efforts alive by funding wildlife protection, supporting local communities, and sustaining fragile ecosystems.

There's a larger truth in this as well. You're just one person — and your choices matter, but they were never meant to bear the full weight of a systemic crisis. You didn't build the fossil fuel-dependent systems that make flying one of the only practical ways to stay connected across long distances or encounter the wider world. Guilt can keep us preoccupied with ourselves rather than on the larger structures driving the harm.

Continue to balance your personal choices with collective ones. To start, you might want to discuss your dilemma with friends and family — whether they're your travel companions, or people in your life who may have already made different calculations about travel. (In last month's column, I shared similar advice with a question asker who could well be on the other side of that conversation .) Having a shared understanding with loved ones might help you process your inner tension, while also letting your care for the planet ripple out in community.

More from Ask a Climate Therapist

Ask a Climate Therapist: How do I deal with friends and family who won't stop polluting?

Take some time to reflect, and then settle into commitments that feel right for you. Return to your decision from time to time and make changes as needed. When you feel a pang of guilt, let it keep you honest but not overwhelmed.

Remember: The places you visit, the people you love across long distances, and the wonder you feel in national parks aren't separate from your love for the Earth. They're expressions of it. Add in a dose of reflection, gratitude, and perhaps even awe, and your journeys will strengthen the passion and resilience that allow you to stay engaged in your work. In this way, the same travel that stirs your guilt can also help protect what you love. It's not a clean or simple solution, but it's real.

The aim is to stay present for the whole of it — to allow your love for our planet to be large enough to hold grief, responsibility, and joy at the same time. Let yourself embrace and enjoy the world wholeheartedly, even as you engage in protecting it.

Holding this with you, Leslie

I'm Leslie Davenport , a licensed therapist, educator, speaker, consultant, and internationally recognized voice on the emotional and psychological dimensions of climate change. If you've got a question about climate and mental health, please submit it here for a future column .

The 12 Bugs of Christmas - A Software Developers' Version

Scott Johnson (fuzzyblog)

This isn't mine but it was forwarded to me via a group chat and, in it, I found a deep, profound and abiding truth:

1. For the first bug of Christmas, my manager said to me:

See if they can do it again.

2. For the second bug of Christmas, my manager said to me:

Ask them how they did it and

See if they can do it again.

3. For the third bug of Christmas, my manager said to me:

Try to reproduce it

Ask them how they did it and

See if they can do it again.

4. For the fourth bug of Christmas, my manager said to me:

Run with the debugger

Try to reproduce it

Ask them how they did it and

See if they can do it again.

5. For the fifth bug of Christmas, my manager said to me:

Ask for a dump

Run with the debugger

Try to reproduce it

Ask them how they did it and

See if they can do it again.

6. For the sixth bug of Christmas, my manager said to me:

Reinstall the software

Ask for a dump

Run with the debugger

Try to reproduce it

Ask them how they did it and

See if they can do it again.

7. For the seventh bug of Christmas, my manager said to me:

Say they need an upgrade

Reinstall the software

Ask for a dump

Run with the debugger

Try to reproduce it

Ask them how they did it and

See if they can do it again.

8. For the eighth bug of Christmas, my manager said to me:

Find a way around it

Say they need an upgrade

Reinstall the software

Ask for a dump

Run with the debugger

Try to reproduce it

Ask them how they did it and

See if they can do it again.

9. For the ninth bug of Christmas, my manager said to me:

Blame it on the hardware

Find a way around it

Say they need an upgrade

Reinstall the software

Ask for a dump

Run with the debugger

Try to reproduce it

Ask them how they did it and

See if they can do it again.

10. For the tenth bug of Christmas, my manager said to me:

Change the documentation

Blame it on the hardware

Find a way around it

Say they need an upgrade

For the fifth bug of Christmas, my manager said to me: Ask for a dump Run with the debugger Try to reproduce it Ask them how they did it and See if they can do it again.

Scott Johnson (fuzzyblog)

Running Multiple Rails Apps Concurrently with Foreman and Procfile.dev

Scott Johnson (fuzzyblog)

Pizza courtesy of Pizza for Ukraine!

Donate Now to Pizza for Ukraine

As I've said, I build a lot of side projects and I really, really like the model of having:

ALL MY APPS RUNNING CONCURRENTLY

I may be a scattered, distracted developer trying to do too damn much but that's my damn right. And I have 64 gigs of RAM so why shouldn't I be this way.

What I want is to be able to switch from app to app and make changes. This is important because some apps provide APIs which other apps rely on and having to figure out what thing is on what port, etc, is just plain distracting.

Foreman and Procfile.dev is a way around this but there's a major hitch in your getalong (as my Texas wife might say).

Here's a sample Procfile.dev for an app I'm building called Cartazzi which makes a developer's life easier:

web: bin/rails server -p 5000 css: yarn build:css –watch js: yarn build –reload # docker: docker-compose up

And here's a Profile.dev for another application called Poolwizard which helps you maintain your swimming pool:

#web: bin/rails server -p $PORT web: bin/rails server -p 5700 css: yarn build:css –watch js: yarn build –reload # docker: docker-compose up worker: bundle exec sidekiq

If you run Cartazzi and Poolwizard together then you're going to get crashes and here's the error:

❯ foreman start -f Procfile.dev

09:11:53 web.1 | started with pid 72877

09:11:53 css.1 | started with pid 72878

09:11:53 js.1 | started with pid 72879

09:11:53 worker.1 | started with pid 72881

09:11:53 js.1 | yarn run v1.22.5

09:11:53 css.1 | yarn run v1.22.5

09:11:53 css.1 | $ tailwindcss –postcss -i ./app/assets/stylesheets/application.tailwind.css -o ./app/assets/builds/application.css –watch

09:11:53 js.1 | $ node esbuild.config.js –reload

09:11:54 js.1 | node:events:371

09:11:54 js.1 | throw er; /\/ Unhandled 'error' event

09:11:54 js.1 | ^

09:11:54 js.1 |

09:11:54 js.1 | Error: listen EADDRINUSE: address already in use :::5200

09:11:54 js.1 | at Server.setupListenHandle [as _listen2] (node:net:1306:16)

09:11:54 js.1 | at listenInCluster (node:net:1354:12)

09:11:54 js.1 | at Server.listen (node:net:1441:7)

09:11:54 js.1 | at buildAndReload (/Users/sjohnson/Sync/coding/rails/poolwizard/esbuild.config.js:54:6)

09:11:54 js.1 | at Object. (/Users/sjohnson/Sync/coding/rails/poolwizard/esbuild.config.js:100:3)

09:11:54 js.1 | at Module._compile (node:internal/modules/cjs/loader:1109:14)

09:11:54 js.1 | at Object. Module._extensions..js (node:internal/modules/cjs/loader:1138:10)

09:11:54 js.1 | at Module.load (node:internal/modules/cjs/loader:989:32)

09:11:54 js.1 | at Function. Module._load (node:internal/modules/cjs/loader:829:14)

09:11:54 js.1 | at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:76:12)

09:11:54 js.1 | Emitted 'error' event on Server instance at:

09:11:54 js.1 | at emitErrorNT (node:net:1333:8)

09:11:54 js.1 | at processTicksAndRejections (node:internal/process/task_queues:83:21) {

09:11:54 js.1 | code: 'EADDRINUSE',

09:11:54 js.1 | errno: −48,

09:11:54 js.1 | syscall: 'listen',

09:11:54 js.1 | address: '::',

09:11:54 js.1 | port: 5200

09:11:54 js.1 | }

09:11:54 js.1 | error Command failed with exit code 1.

09:11:54 js.1 | info Visit https://yarnpkg.com/en/docs/cli/run for documentation about this command.

09:11:54 js.1 | exited with code 1

Exporting Node Modules in 2022

David Bushong · Groupon Engineering

Groupon maintains literally hundreds of NPM modules, both open source and internal. Many of these are consumed by our custom NodeJS-based middleware web layer we call “The Interaction Tier” (itself a topic for another post someday).

As folks write new modules, a common question is “what’s the best way to export things from our published modules to maximize compatibility?” — and that is the topic of this post. First, some background and history:

Flavors of exported modules

CommonJS in Node

In the beginning, there was CommonJS :

Here are 3 sample files with exports:

/\/ export1.js - exporting individual properties w/ CommonJS ‘use strict’; function foo() { } exports.foo = foo; exports.bar = 42; /\/ export2.js - exporting a single object w/ CommonJS ‘use strict’; function baz() { } const garply = 88; module.exports = { baz }; /\/ dynamically (conditionally!) exported! if (Math.random() > 0.5) module.exports.garply = garply; /\/ export3.js - exporting a bare function w/ CommonJS ‘use strict’; function quux() { } module.exports = quux; /\/ sometimes there are extra properties added to the bare function quux.yadda = 42;

Those CommonJS exports can be imported either into other CommonJS files or into ES Module (more on that below) files:

/\/ import.js - importing CommonJS modules into a CJS file ‘use strict’; const { foo, bar } = require(‘./export1’); const { baz, garply } = require(‘./export2’); if (Math.random > 0.9) { /\/ can also dynamically decide when to import const quux = require(‘./export3’); /\/ can poke into properties tacked onto functions const { yadda } = require(‘./export3’); } /\/ import.mjs - importing CommonJS modules into an ESM file /\/ node is willing to turn exported objects into named exports import { foo, bar } from ‘./export1.js’; import { baz, garply } from ‘./export2.js’; import quux from ‘./export3.js’; /\/ cannot access added property “yadda” directly; hence: const { yadda } = quux;

You can read more elsewhere, but the key features are:

The module files are synchronously executed, and at the end of their sync execution, anything present in module.exports is available to files that require() this module. This means exports can be dynamically constructed.

You may either add properties to the existing exports object, which starts the same as module.exports, or reassign all of module.exports to a single thing (which may happen to be an object)

The reason const { a, b } = require(... works is because it is a convention to export an object - but you can export anything you want (bare function, array, number, etc)

Native ES Modules in Node

This is a large topic , but here are the highlights:

First, three example ES Modules-exporting files:

/\/ export1.mjs - named exports w/ ESM export function foo() { } export const bar = 42; /\/ export2.mjs - more named exports w/ ESM export function baz() { } /\/ cannot dynamically choose to not export things /\/ i.e. no if (...) export possible export const garply = 88; /\/ export3.mjs - default export w/ ESM export default function quux() { } /\/ can export something *other* than default also export const yadda = 99;

ES Module exports can be imported easily in other ES Module files, and can be imported asynchronously (only!) in CommonJS files:

/\/ import.mjs - importing ESM w/ ESM import { foo, bar } from ‘./export1.mjs’; import { baz, garply } from ‘./export2.mjs’; /\/ can import default export and others import quux, { yadda } from ‘./export3.mjs’; /\/ import.js - importing ESM w/ CommonJS ‘use strict’; /\/ you cannot directly require() ESM files, /\/ you must use import() which is an async operator async function someFn() { const { foo, bar } = await import (‘./export1.mjs’); const { baz, garply } = await import (‘./export2.mjs’); /\/ the default export has property “default” const { default: quux, yadda } = await import (‘./export3.mjs’); }

BabelScript / Webpack / TypeScript

When early proposals of ECMAScript Modules were announced, a number of bundlers (and TypeScript) ran with it, and supported various syntaxes which were similar to, but not entire semantically compatible with, ES Modules as ultimately supported natively in NodeJS 14.

Most projects generally compile these files to CommonJS, so we should understand what they actually compile to. export1.mjs and export2.mjs would generally compile to their equivalents from the CommonJS section above. export3.mjs would compile to something like:

/\/ dist/export3.js ‘use strict’; function quux() { } exports.default = quux; exports.yadda = 99;

This is importantly different from the original CommonJS export3.js, because there is no bare function exported, but rather an object with property default. To maintain backward compatibility when required, TypeScript offers a non-standard syntax:

/\/ export3.ts function quux() { } export = quux; /\/ non-standard syntax /\/ cannot export anything else, like yadda, though you can: quux.yadda = 99;

cPouta - Now with Haka!

Kalle Happonen · CSC Cloud Team Blog

Many times we've been asked

Can I log in to cPouta using my Haka account?

Finally we can answer yes!

"How do I log in again?"

Not THAT kind of haka.

...but

Things are never quite that simple. There are a few restrictions. The most important one is, that you have to have a CSC account and access to a cPouta project to do this.

How do you get these? This page should get you started.

https://research.csc.fi/accounts-and-projects

We still have to keep track of research projects, their users, and their resource usage, which is why you still need a project. But once you have that, point your browser to

https://pouta.csc.fi

and log in to your heart's content.

Please note that Haka logins only work with the web interface for now. We're looking into the option of enabling this for API and command line access in the future.

Your CSC Cloud Team

Turning Off Ruby Deprecation Warnings When Running Tests

Scott Johnson (fuzzyblog)

The combination of Rails 6 and Ruby 2.7, or perhaps just Rails 6, introduced some new deprecation warnings and, lately, I've been seeing this cruft constantly:

rails test test/models/

Running via Spring preloader in process 34024

/Users/sjohnson/.rvm/gems/ruby-2.7.0/gems/activemodel-6.0.2.1/lib/active_model/type/integer.rb:13: warning: Using the last argument as keyword parameters is deprecated; maybe ** should be added to the call

/Users/sjohnson/.rvm/gems/ruby-2.7.0/gems/activemodel-6.0.2.1/lib/active_model/type/value.rb:8: warning: The called method `initialize' is defined here

/Users/sjohnson/.rvm/gems/ruby-2.7.0/gems/activerecord-6.0.2.1/lib/active_record/connection_adapters/postgresql/oid/specialized_string.rb:12: warning: Using the last argument as keyword parameters is deprecated; maybe ** should be added to the call

/Users/sjohnson/.rvm/gems/ruby-2.7.0/gems/activemodel-6.0.2.1/lib/active_model/type/value.rb:8: warning: The called method `initialize' is defined here

And, in the immortal words of Twisted Sister , we're not going to take it anymore.

The solution is a bit more byzantine than I have found in the Rails world and a bunch of the options in the Stack Overflow "top of Google" but no longer accurate post no longer work or aren't quite what you expect.

The easy solution is to add:

$VERBOSE=nil

to the top of config/environments/test.rb so it looks like this:

$VERBOSE = nil

Rails. applica-

IN BRIEF

Scott Johnson (fuzzyblog), Scott Johnson (fuzzyblog): This falls into the category of "I'm old and can't remember this so must, must, must write it down" because I keep losing this html page . If you need to change the text on a submit button for a Rails form implemented with simple_form then use:

rb:8: warning: The called method `initialize' is defined here And, in the immortal words of Twisted Sister , we're not going to take it anymore.

Scott Johnson (fuzzyblog)

Latent Effects for Reusable Language Components

Lambda the Ultimate

The development of programming languages can be quite complicated and costly. Hence, much effort has been devoted to the modular definition of language features that can be reused in various combinations to define new languages and experiment with their semantics. A notable outcome of these efforts is the algebra-based "datatypes "a la carte" (DTC) approach. When combined with algebraic effects, DTC can model a wide range of common language features. Unfortunately, the

current state of the art does not cover modular definitions of advanced control-flow mechanisms that defer execution to an appropriate point, such as call-by-name and call-by-need evaluation, as well as (multi-)staging. This paper defines latent effects, a generic class of such control-flow mechanisms. We demonstrate how function abstractions, lazy computations and a MetaML-like staging can all be expressed in a modular fashion using latent effects, and how they can be combined in various ways to obtain complex semantics. We provide a full Haskell implementation of our effects and handlers with a range of examples.

Looks like a nice generalization of the basic approach taken by algebraic effects to more subtle contexts. Algebraic effects have been discussed here on LtU many times . I think this description from section 2.3 is a pretty good overview of their approach:

LE&H is based on a different, more sophisticated structure than AE&H's free monad. This structure supports non-atomic operations (e.g., function abstraction, thunking, quoting) that contain or delimit computations whose execution may be deferred. Also, the layered handling is different. The idea is still the same, to replace bit by bit the structure of the tree by its meaning. Yet, while AE&H grows the meaning around the shrinking tree, LE&H grows little "pockets of meaning" around the individual nodes remaining in the tree, and not just around the root. The latter supports deferred effects because later handlers can still re-arrange the semantic pockets created by earlier handlers.

Coding with Python Dictionaries the Ruby Way With Respect to Missing Keys

Scott Johnson (fuzzyblog)

I recently had an interesting experience with Python dictionaries that made question one of Python's foundational principles i.e. TOOWTDI:

There should be one– and preferably only one –obvious way to do it. Although that way may not be obvious at first unless you're Dutch. More...

The application was a large scale JSON processor and, in Python, JSONs are mapped to Python dictionaries which correspond to Ruby hashes. The issue is how you handle keys that don't exist. Here's the Ruby way:

irb (main): 001 : 0 > hash = {} {} irb (main): 002 : 0 > hash ["foo"] = "bar" "bar" irb (main): 003 : 0 > hash ["blah"] nil

And here's the Python way:

>>> dict = {} >>> dict ["foo"] = "bar" >>> dict ["blah"] Traceback (most recent call last): File " " , line 1 , in KeyError : 'blah' >>>

At the heart of things is whether or not a non-existent key causes an exception to be thrown. I look at this from the cultural perspective of the language's creators.

Ruby says:

"Oh hell no; good heavens why would a non existent key cause an error. We're Japanese at our heart and tossing an exception for this feels, well, ahem, rude."

While Python says:

"Absolutely a missing key must translate to an exception; we are Dutch after all!"

The interesting thing here is that despite the foundational principle of TOOWTDI, Python offers a way to emulate Ruby's default behavior – the .get() syntax. Here's an example:

>>> dict = {} >>> dict ["foo"] = "bar" >>> dict . get ("foo") 'bar' >>> dict . get ("blah") >>> result = dict . get ("blah") >>> print (result) None

My vastly more Pythonic co-worker, Grant, gave me this explanation when I raised this issue in our team Slack:

@sjohnson You can make Python dicts return None if the key doesn't exist by using the .get class function instead of the square-brackets key notation. So if d is your dictionary, you can use d.get(key) instead of d[key]. The latter will throw a KeyError if the key doesn't exist, whereas the former will just return a null value.

Thanks Grant!

A Regular Expression for Validating An Internet Domain

Scott Johnson (fuzzyblog)

Despite the power and truth of Jamie Zawinski's law:

Regular Expressions: Now You Have Two Problems Jeff Atwood's Perspective

Like Jeff, I too really, really love regular expressions or regexes. I use this one a lot and I finally learned to use \S (Any non-whitespace character) so here's a regex

^\S+\.\S+$

that I wrote yesterday to "validate" the permitted characters in an Internet domain. I was all proud of this and wrote this blog post only to realize that pride really does goeth before a fall – this will NOT correctly validate an Internet domain. As I write this post, I realize that the number of allowed characters in an Internet domain are actually NOT any non-whitespace characters and here's the proof that I actually got that wrong yesterday when I put something online using it:

Note : The fact that Rubular allows through an _ which is NOT a valid character in domains is problematic.

So the right way to do this, DAMN IT, is something like this:

^[A-Za-z0-9\-]+\.[A-Za-z0-9\-]+$

And this actually works:

The [A-Za-z0-9-]+ is a "character class" which says "Any uppercase or lowercase letter plus 0-9 plus a -" are allowed (any order, any quantity)".

Regular Expressions – Now you have two problems.

The Gazette

Vol. 1, No. 008 · 2026-03-30

Curated and composed automatically.